

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Alfin Hadi Komarudin - 5024241069

2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Mengatur konfigurasi `platformio.ini` untuk board STM32 Blackpill menggunakan framework Arduino dan ST-Link sebagai upload protocol.

2. Menghubungkan board STM32 Blackpill ke ST-Link melalui pin SWDIO, SWCLK, GND, dan 3.3V.

3. Menuliskan program Blink pada file `src/main.cpp` dengan kode berikut.

```
#include <Arduino.h>

#define LED_PIN PC13

void setup()
{
  pinMode(LED_PIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_PIN, LOW);
  delay(500);
  digitalWrite(LED_PIN, HIGH);
  delay(500);
}
```

4. Melakukan Build dan Upload menggunakan PlatformIO, kemudian mengamati LED onboard pada pin PC13.

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Mengatur konfigurasi `platformio.ini` untuk board ESP32 DevKit menggunakan framework ESP-IDF.

2. Menghubungkan board ESP32 ke komputer melalui kabel USB.

3. Membuat file `src/main.c` dan menuliskan program Blink menggunakan framework ESP-IDF sebagai berikut.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void)
{
    gpio_reset_pin(LED_PIN);
    gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

    while (1)
```

```
    {
        gpio_set_level(LED_PIN, 1);
        vTaskDelay(pdMS_TO_TICKS(500));
        gpio_set_level(LED_PIN, 0);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

4. Melakukan Build dan Upload program, kemudian mengamati LED onboard ESP32 yang berkedip secara berulang.

1.3 Percobaan 3: Pull-up dan Pull-down Eksternal vs Internal (ESP32 + Framework Arduino)

1.3.1 Eksperimen 1: Pull-up dan Pull-down Eksternal

1. Mengatur konfigurasi `platformio.ini` menggunakan board ESP32 dengan framework Arduino.
2. Merangkai dua pushbutton pada breadboard menggunakan resistor eksternal 10 kΩ. Rangkaian pull-up dihubungkan ke GPIO 32, sedangkan pull-down dihubungkan ke GPIO 33.
3. Menuliskan program pembacaan kondisi kedua pushbutton pada file `src/main.cpp` sebagai berikut.

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32
#define BUTTON_PULLDOWN_PIN 33

void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PULLUP_PIN, INPUT);
    pinMode(BUTTON_PULLDOWN_PIN, INPUT);
}

void loop() {
    int stateUp = digitalRead(BUTTON_PULLUP_PIN);
    int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

    Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                  stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                  stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
    delay(200);
}
```

4. Melakukan Build dan Upload program, kemudian mengamati perubahan kondisi logika tombol melalui Serial Monitor ketika tombol ditekan dan dilepaskan.

2 Analisis Hasil Percobaan

Pada praktikum ini dilakukan pengujian penggunaan GPIO pada mikrokontroler STM32 Blackpill dan ESP32. Pengujian meliputi konfigurasi GPIO sebagai output untuk mengendalikan LED menggunak-

an framework Arduino dan ESP-IDF, serta konfigurasi GPIO sebagai input melalui rangkaian pull-up dan pull-down eksternal. Melalui percobaan tersebut, dapat dipahami bagaimana mikrokontroler memproses sinyal digital untuk berinteraksi dengan perangkat keras yang terhubung.

Pada percobaan pertama, board STM32 Blackpill digunakan untuk menjalankan program Blink menggunakan framework Arduino. Konfigurasi pada file `platformio.ini` digunakan untuk menentukan board dan framework, sedangkan ST-Link berfungsi untuk mengunggah program melalui antarmuka SWD. Pin PC13 dikonfigurasikan sebagai output menggunakan fungsi `pinMode()`, kemudian dikendalikan menggunakan fungsi `digitalWrite()`. Fungsi `delay(500)` memberikan jeda selama 500 ms pada setiap perubahan kondisi logika pin.

LED onboard STM32 Blackpill memiliki karakteristik aktif-LOW, sehingga kondisi LOW menyebabkan LED menyala dan kondisi HIGH menyebabkan LED mati. Program mengatur pin PC13 pada kondisi LOW selama 500 ms, kemudian mengubahnya menjadi HIGH selama 500 ms secara berulang. Dengan demikian, LED mengalami proses menyala dan mati secara bergantian dengan periode satu detik. Percobaan ini menunjukkan bahwa GPIO dapat digunakan sebagai output digital untuk mengendalikan perangkat berdasarkan instruksi program.

Pada percobaan kedua, pengujian Blink dilakukan menggunakan ESP32 DevKit dengan framework ESP-IDF. Program menggunakan fungsi `gpio_reset_pin()` untuk mengatur ulang konfigurasi pin dan `gpio_set_direction()` untuk menetapkan GPIO 2 sebagai output. Selanjutnya, fungsi `gpio_set_level()` digunakan untuk mengubah kondisi logika pin menjadi HIGH dan LOW secara bergantian. Pengaturan jeda dilakukan menggunakan `vTaskDelay(pdMS_TO_TICKS(500))` yang memanfaatkan FreeRTOS untuk memberikan jeda selama 500 ms pada setiap perubahan kondisi.

Jika dibandingkan dengan percobaan pertama, kedua program memiliki tujuan yang sama, yaitu mengendalikan LED melalui GPIO sebagai output digital. Perbedaannya terletak pada fungsi dan pendekatan pemrograman yang digunakan. Framework Arduino menyediakan fungsi seperti `pinMode()`, `digitalWrite()`, dan `delay()`, sedangkan ESP-IDF menggunakan fungsi yang lebih spesifik untuk mengatur peripheral GPIO serta memanfaatkan FreeRTOS dalam pengaturan waktu eksekusi. Hal ini menunjukkan bahwa pengendalian perangkat keras dapat dilakukan melalui framework yang berbeda dengan mekanisme pemrograman masing-masing.

Pada percobaan ketiga, dilakukan pengujian GPIO sebagai input menggunakan rangkaian pull-up dan pull-down eksternal dengan resistor 10 kΩ. Rangkaian pull-up dihubungkan ke GPIO 32 dengan resistor menuju sumber tegangan 3,3 V, sehingga kondisi awal pin berada pada logika HIGH. Ketika tombol ditekan, pin terhubung ke GND dan berubah menjadi LOW. Sebaliknya, rangkaian pull-down dihubungkan ke GPIO 33 dengan resistor menuju GND, sehingga kondisi awal pin berada pada logika LOW. Ketika tombol ditekan, pin terhubung ke sumber tegangan 3,3 V dan berubah menjadi HIGH.

Pembacaan kondisi kedua tombol dilakukan menggunakan fungsi `digitalRead()`, kemudian hasilnya ditampilkan melalui Serial Monitor menggunakan `Serial.printf()`. Program mengidentifikasi tombol pull-up sebagai kondisi ditekan ketika nilai GPIO terbaca LOW, sedangkan tombol pull-down dianggap ditekan ketika nilai GPIO terbaca HIGH. Penggunaan resistor eksternal berfungsi menen-

tukan kondisi logika awal pada pin input dan mencegah terjadinya kondisi mengambang atau *floating* ketika tombol tidak ditekan. Dengan demikian, perubahan kondisi tombol dapat dibaca berdasarkan nilai logika yang lebih terarah.

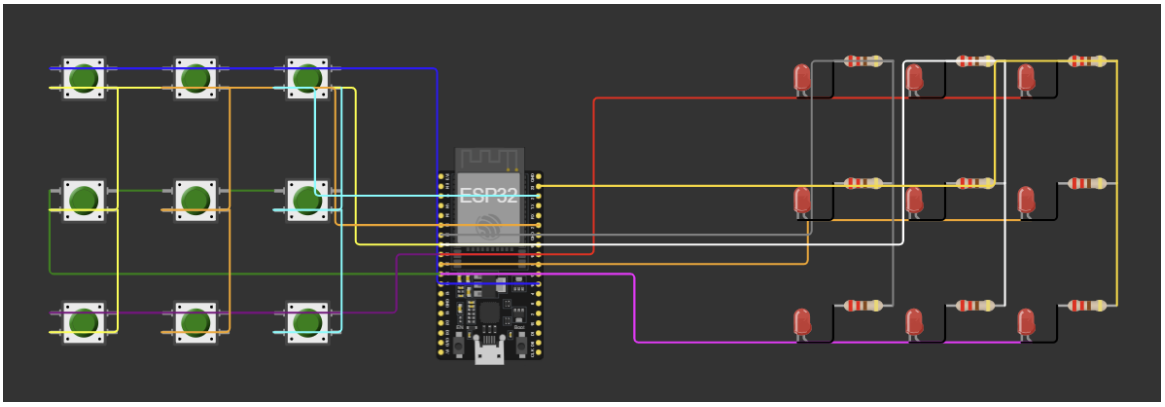
Secara keseluruhan, praktikum ini menunjukkan perbedaan penggunaan GPIO sebagai output dan input pada mikrokontroler. Pengujian Blink memperlihatkan bagaimana sinyal digital digunakan untuk mengendalikan LED, sedangkan pengujian pull-up dan pull-down memperlihatkan bagaimana mikrokontroler membaca masukan dari pushbutton. Selain itu, praktikum ini memberikan pemahaman mengenai perbedaan penggunaan framework Arduino dan ESP-IDF serta pentingnya konfigurasi rangkaian eksternal dalam menjaga kestabilan sinyal digital pada sistem tertanam.

3 Hasil Tugas Modul

3.1 Hasil Simulasi dan Source Code Matrix Pushbutton 3x3 yang Mengendalikan Matriks LED 3x3

3.1.1 Hasil Simulasi Rangkaian

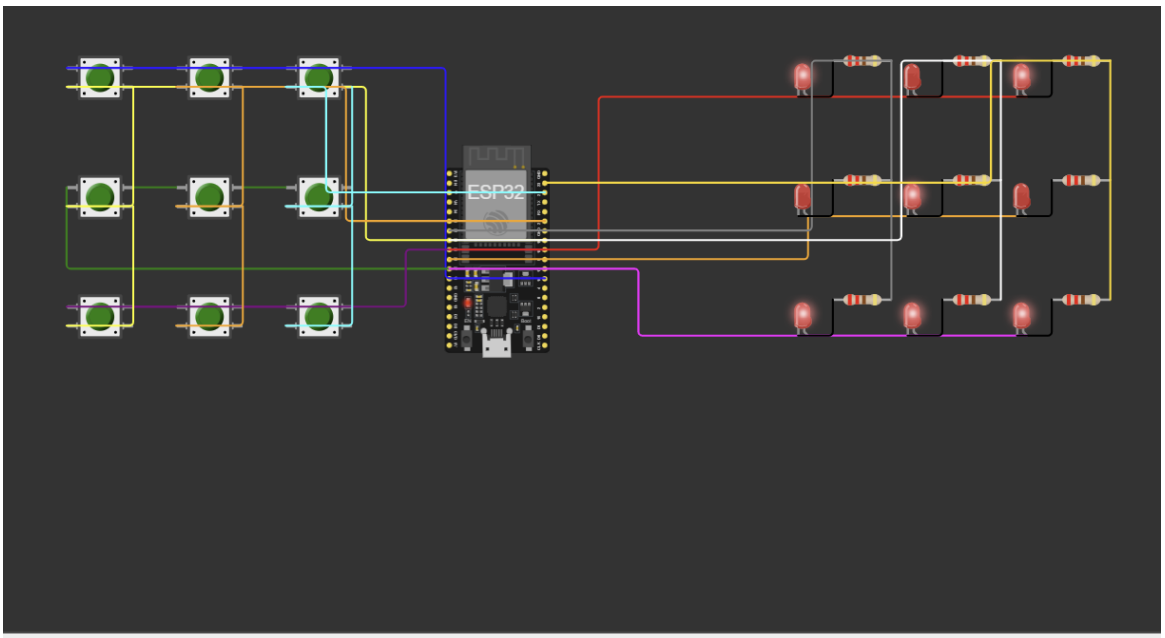
Rangkaian simulasi dibuat menggunakan aplikasi Wokwi dengan mikrokontroler ESP32. Rangkaian terdiri dari sembilan pushbutton yang disusun dalam matriks 3x3 dan sembilan LED yang disusun dengan pola yang sama. Setiap pushbutton digunakan untuk mengendalikan LED pada posisi yang bersesuaian.



Gambar 1: Rangkaian Simulasi Matrix Pushbutton dan LED 3x3

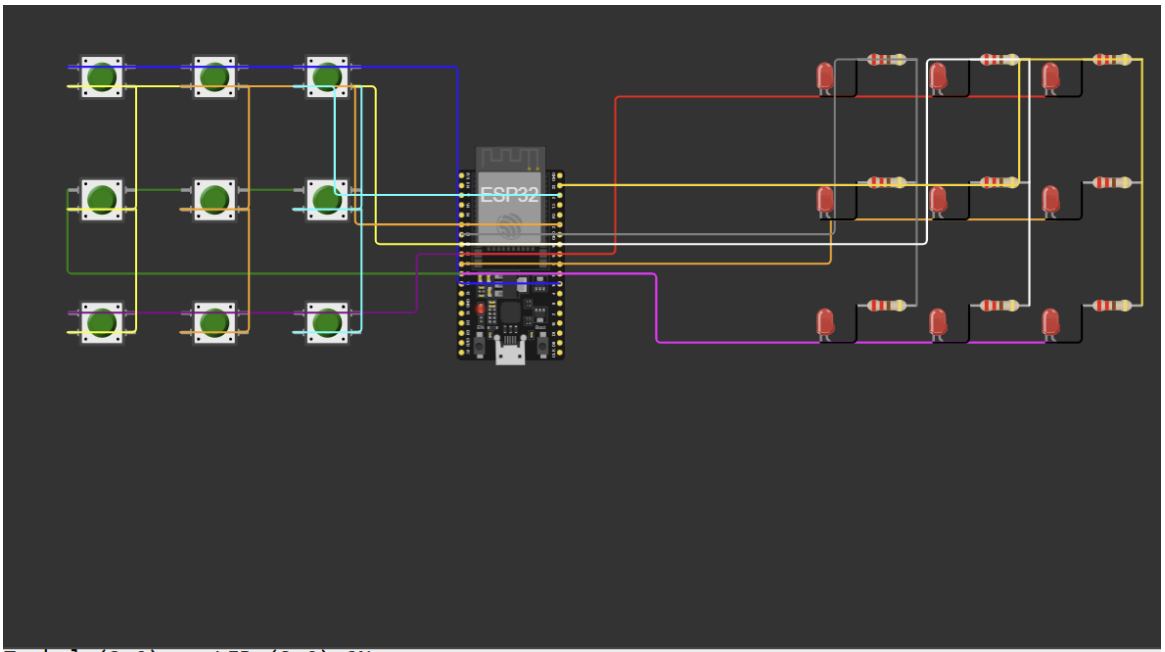
3.1.2 Hasil Pengujian Simulasi

Setelah rangkaian dan program dijalankan, dilakukan pengujian dengan menekan beberapa pushbutton. Setiap tombol yang ditekan akan mengubah status LED pada posisi yang sama. Hasil perubahan kondisi LED dapat diamati secara langsung melalui simulasi dan keluaran pada Serial Monitor.




```
Matrix 3x3. Tekan tombol untuk menyalakan LED.
Tombol (1,1) -> LED (1,1) ON
Tombol (2,2) -> LED (2,2) ON
Tombol (3,3) -> LED (3,3) ON
Tombol (1,3) -> LED (1,3) ON
Tombol (3,1) -> LED (3,1) ON
Tombol (3,2) -> LED (3,2) ON
```

Gambar 2: Hasil ON pada Pengujian Simulasi Matrix Pushbutton dan LED 3x3




```
Tombol (3,2) -> LED (3,2) ON
Tombol (1,1) -> LED (1,1) OFF
Tombol (1,3) -> LED (1,3) OFF
Tombol (2,2) -> LED (2,2) OFF
Tombol (3,1) -> LED (3,1) OFF
Tombol (3,2) -> LED (3,2) OFF
Tombol (3,3) -> LED (3,3) OFF
```

Gambar 3: Hasil OFF pada Pengujian Simulasi Matrix Pushbutton dan LED 3x3

3.1.3 Source Code

Program berikut digunakan untuk membaca input dari matriks pushbutton, menerapkan debouncing, dan mengendalikan matriks LED berdasarkan posisi tombol yang ditekan.

```
#include <Arduino.h>

const int buttonRows[3] = {16, 17, 18};
const int buttonCols[3] = {19, 21, 22};

const int ledRows[3] = {25, 26, 27};
const int ledCols[3] = {32, 33, 23};

bool ledState[3][3] = {};
bool rawState[3][3] = {};
bool stableState[3][3] = {};
unsigned long lastDebounceTime[3][3] = {};

const unsigned long debounceDelay = 30;

void scanButtons() {
    for (int r = 0; r < 3; r++) {
        for (int i = 0; i < 3; i++) {
            digitalWrite(buttonRows[i], HIGH);
        }

        digitalWrite(buttonRows[r], LOW);
        delayMicroseconds(50);

        for (int c = 0; c < 3; c++) {
            bool reading = digitalRead(buttonCols[c]) == LOW;

            if (reading != rawState[r][c]) {
                rawState[r][c] = reading;
                lastDebounceTime[r][c] = millis();
            }

            if (millis() - lastDebounceTime[r][c] >= debounceDelay) {
                if (stableState[r][c] != rawState[r][c]) {
                    stableState[r][c] = rawState[r][c];

                    if (stableState[r][c]) {
                        ledState[r][c] = !ledState[r][c];

                        Serial.printf(
                            "Tombol (%d,%d) -> LED (%d,%d) %s\n",
                            r + 1, c + 1, r + 1, c + 1,
                            ledState[r][c] ? "ON" : "OFF"
                        );
                    }
                }
            }
        }
    }

```

```
51      for (int i = 0; i < 3; i++) {
52          digitalWrite(buttonRows[i], HIGH);
53      }
54  }
55
56  void refreshLED() {
57      for (int r = 0; r < 3; r++) {
58          for (int i = 0; i < 3; i++) {
59              digitalWrite(ledRows[i], LOW);
60          }
61
62          for (int c = 0; c < 3; c++) {
63              digitalWrite(ledCols[c], HIGH);
64          }
65
66          digitalWrite(ledRows[r], HIGH);
67
68          for (int c = 0; c < 3; c++) {
69              digitalWrite(
70                  ledCols[c],
71                  ledState[r][c] ? LOW : HIGH
72              );
73          }
74
75          delay(2);
76      }
77
78      for (int r = 0; r < 3; r++) {
79          digitalWrite(ledRows[r], LOW);
80      }
81  }
82
83  void setup() {
84      Serial.begin(115200);
85
86      for (int r = 0; r < 3; r++) {
87          pinMode(buttonRows[r], OUTPUT);
88          digitalWrite(buttonRows[r], HIGH);
89
90          pinMode(ledRows[r], OUTPUT);
91          digitalWrite(ledRows[r], LOW);
92      }
93
94      for (int c = 0; c < 3; c++) {
95          pinMode(buttonCols[c], INPUT_PULLUP);
96
97          pinMode(ledCols[c], OUTPUT);
98          digitalWrite(ledCols[c], HIGH);
99      }
100
101     Serial.println("Matrix 3x3 siap. Tekan tombol untuk toggle LED.");
102 }
103
104 void loop() {
```

```
105     scanButtons();
106     refreshLED();
107 }
```

3.2 Pertanyaan Analisis

1. Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing.

   Debouncing tetap diperlukan karena tombol mekanis dapat mengalami bouncing, yaitu kondisi ketika kontak di dalam tombol bergetar sesaat saat ditekan atau dilepaskan sehingga menghasilkan beberapa perubahan sinyal HIGH dan LOW dalam waktu singkat. Meskipun hanya satu tombol yang ditekan, bouncing dapat menyebabkan mikrokontroler mendeteksi beberapa kali penekanan. Akibatnya, LED dapat mengalami perubahan kondisi berulang kali. Debouncing digunakan untuk memastikan bahwa perubahan sinyal yang terbaca benar-benar merupakan satu penekanan tombol.

2. Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?

   Pada rangkaian ini digunakan konfigurasi pull-up internal (`INPUT_PULLUP`) pada pin kolom tombol. Ketika tombol tidak ditekan, pin berada pada kondisi HIGH karena terhubung secara internal ke tegangan 3,3 V melalui resistor pull-up. Ketika tombol ditekan dan baris yang bersesuaian diaktifkan LOW, pin kolom akan terbaca LOW. Kondisi pin tidak boleh dibiarkan floating karena tegangan input menjadi tidak memiliki nilai logika yang pasti sehingga dapat menghasilkan pembacaan HIGH atau LOW secara tidak stabil dan menyebabkan deteksi tombol yang keliru.

3. Andaikan salah satu Tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai **digital input** (tombol, hanya HIGH/LOW) dan sebagai **analog input** (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?

   GPIO sebagai digital input hanya membaca dua kondisi logika, yaitu HIGH dan LOW. Pada tombol, kedua kondisi tersebut digunakan untuk menunjukkan apakah tombol sedang ditekan atau tidak. Sementara itu, analog input melalui ADC dapat membaca variasi tegangan dalam rentang tertentu dan mengubahnya menjadi nilai digital berdasarkan resolusi ADC. Oleh karena itu, ADC dapat merepresentasikan lebih dari dua kondisi, misalnya tingkat intensitas cahaya yang berbeda pada LDR, karena setiap perubahan tegangan dapat menghasilkan nilai pembacaan yang berbeda.

4 Kesimpulan

Praktikum ini memberikan pemahaman dasar mengenai penggunaan mikrokontroler STM32 dan ESP32, mulai dari konfigurasi perangkat, pengunggahan program, hingga penerapan fungsi GPIO sebagai input dan output. Selain itu, praktikan juga memperoleh pemahaman mengenai penerapan konfigurasi pull-up dan pull-down pada rangkaian pushbutton, serta bagaimana mikrokontroler

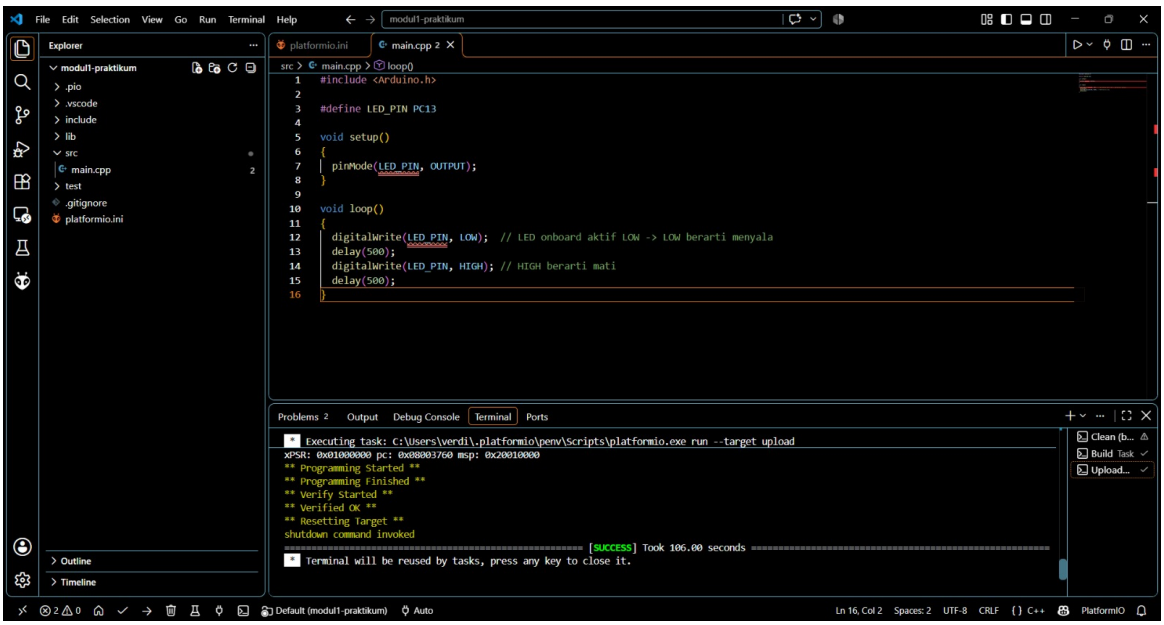
membaca perubahan kondisi logika HIGH dan LOW untuk mengendalikan suatu keluaran. Melalui kegiatan praktikum dan tugas modul, praktikan dapat memahami hubungan antara perangkat keras dan perangkat lunak dalam membangun sistem tertanam sederhana. Namun, pelaksanaan praktikum belum dapat mencakup seluruh percobaan yang tersedia. Terdapat beberapa percobaan yang tidak sempat dilakukan karena kendala teknis pada tahap awal Percobaan 1 dan 2, terutama dalam konfigurasi serta penggunaan PlatformIO untuk menjalankan program pada mikrokontroler. Proses penyelesaian kendala tersebut membutuhkan waktu yang cukup lama sehingga waktu yang tersedia tidak mencukupi untuk melanjutkan seluruh rangkaian percobaan. Kondisi ini menjadi bahan evaluasi bagi kelompok kami agar lebih mempersiapkan lingkungan pengembangan, khususnya dengan memastikan PlatformIO telah terpasang dan terkonfigurasi dengan baik sebelum praktikum dimulai. Dengan persiapan tersebut, diharapkan kendala serupa dapat diminimalkan sehingga waktu praktikum berikutnya dapat dimanfaatkan secara lebih efektif dan seluruh percobaan dapat diselesaikan sesuai dengan tujuan yang telah ditentukan.

5 Lampiran

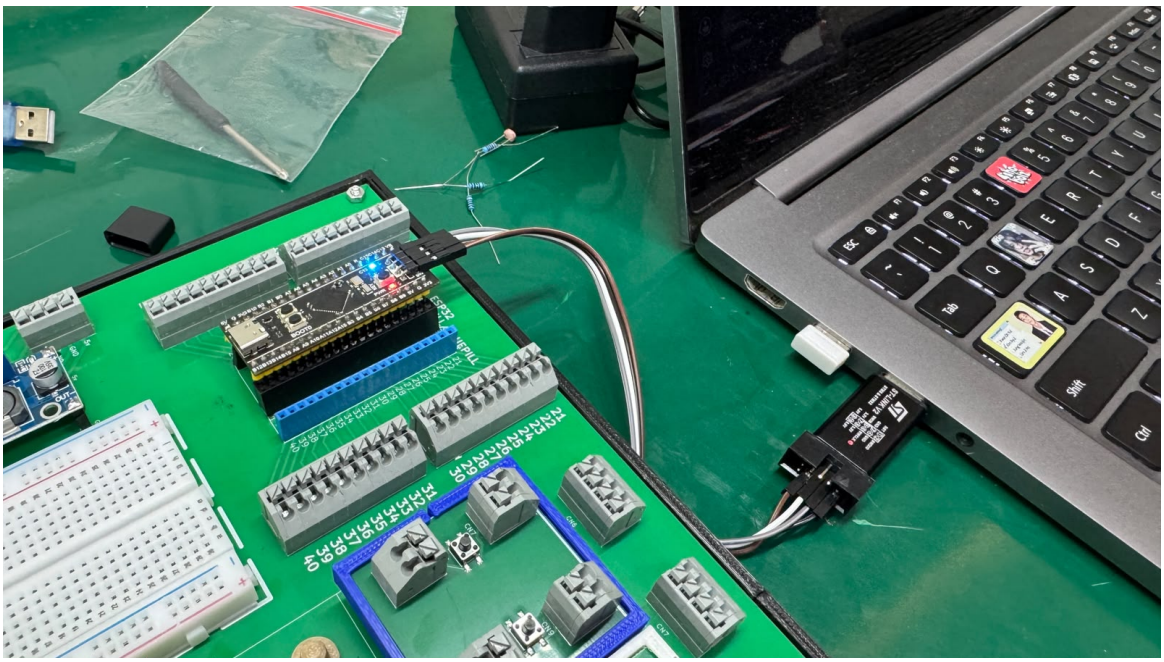
5.1 Dokumentasi saat praktikum



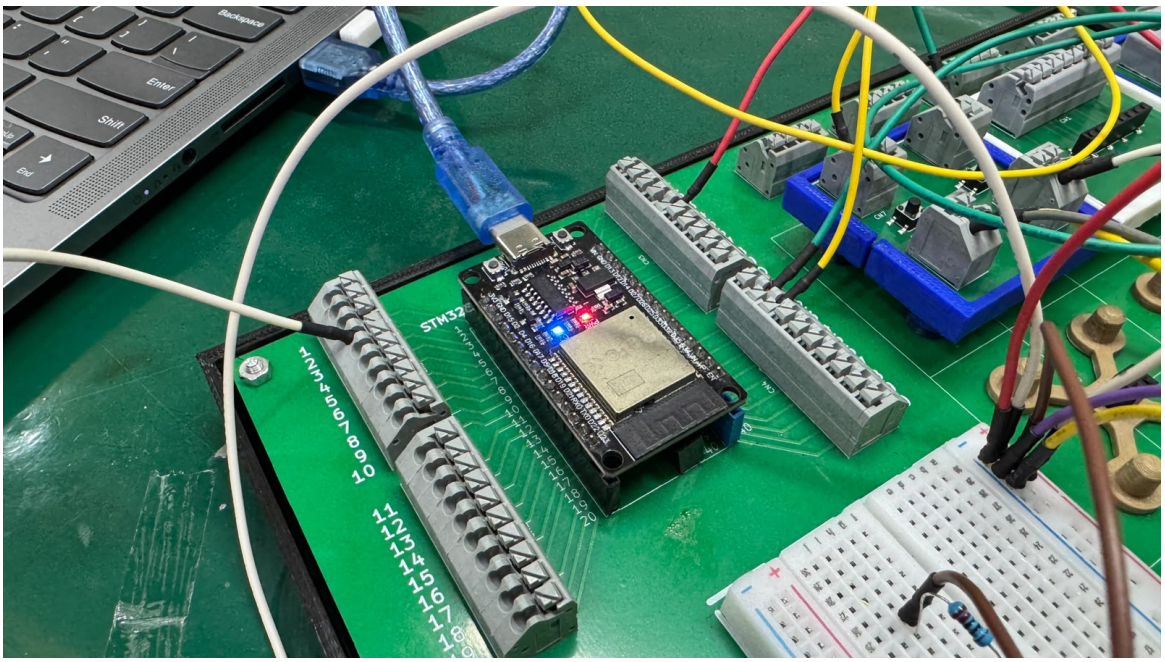
Gambar 4: Komponen STM32 Blackpill yang Siap Dihubungkan ke Laptop



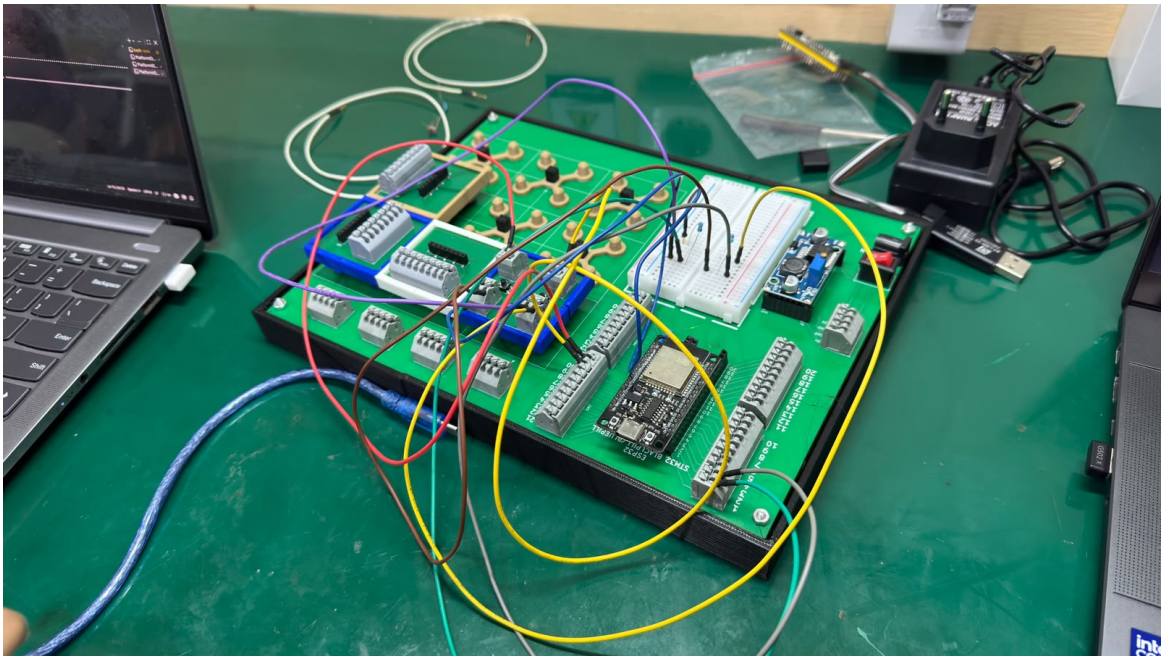
Gambar 5: Hasil Build dan Upload Program STM32 yang Berhasil



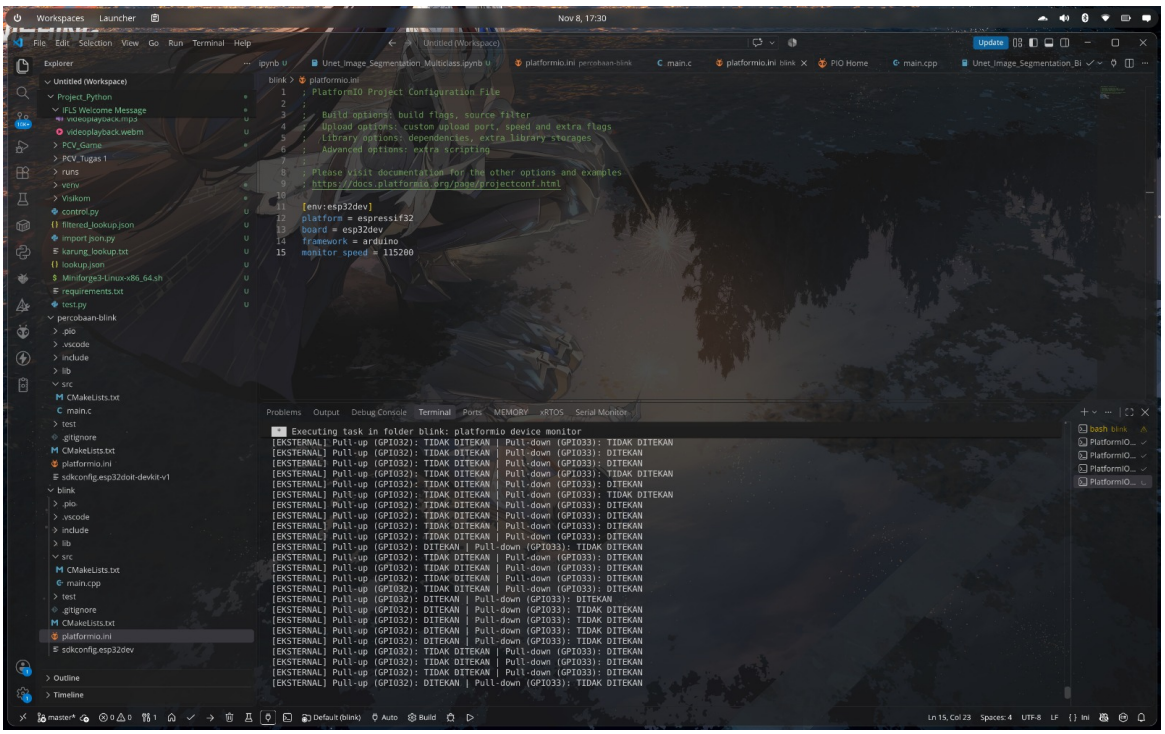
Gambar 6: Hasil Percobaan Blink pada STM32 Blackpill



Gambar 7: Hasil Percobaan Blink pada ESP32



Gambar 8: Rangkaian Percobaan 3 Menggunakan Pull-up Eksternal



Gambar 9: Hasil Build dan Upload Program Percobaan 3 dengan Pull-up Eksternal